# SuperMinHash

## A New Minwise Hashing Algorithm for Jaccard Similarity Estimation

Vicente Hernández  Vicente Wiederhold

Análisis de Grandes Volúmenes de Datos

6 de noviembre de 2026

**Autor:** Otmar Ertl · **Año:** 2017
**Publicación:** preprint en arXiv (arXiv:1706.05698 [cs.DS]);
no registra publicación en revista o conferencia

# Problema y motivación

## Problema que aborda el paper

Construir firmas compactas de conjuntos que permitan estimar su similitud de Jaccard con poco tiempo y memoria, incluso cuando hay muchos conjuntos o estos no caben completos en memoria.

- MinHash estima Jaccard comparando firmas, pero construir una firma requiere $O(nm)$ operaciones: $n$ elementos y $m$ posiciones.
- Al procesar grandes colecciones, este costo se repite para cada conjunto y se vuelve un cuello de botella.
- **Motivación:** reducir el costo y mejorar la precisión, manteniendo una sola pasada sobre los datos.

**Aplicación:** búsqueda de similitud y deduplicación de documentos representados como conjuntos de shingles.

# El punto de partida: Jaccard y MinHash

**Similitud de Jaccard**

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

- $A, B$ son conjuntos de elementos (p. ej., shingles); $J = 0$: disjuntos, $J = 1$: idénticos.

**MinHash**

$$h_j(D) = \min_{d \in D} r_j(d), \quad j = 0, \ldots, m-1$$

- $r_j$: hashes independientes y uniformes en $[0, 1)$; $m$: tamaño de la firma.

**Estimador**

$$\widehat{J} = \frac{1}{m} \sum_{j=0}^{m-1} \mathbf{1}[h_j(A) = h_j(B)]$$

- Insesgado: $\mathbb{E}[\widehat{J}] = J$.
- Varianza: $\mathrm{Var}(\widehat{J}) = \dfrac{J(1-J)}{m}$.
- Entradas $h_0, \ldots, h_{m-1}$ independientes.
- Costo: $O(nm)$, con $n = |D|$.

# Por qué buscar una alternativa

- **MinHash:** $O(nm)$ por firma. Con $m$ grande, el recorrido interno domina el costo.
- **One Permutation Hashing** (Li, Owen y Zhang, 2012): $O(n+m)$, pero con $|D| < m$ muchos bins quedan vacíos. La densificación recupera el estimador, pero pierde precisión; además, tras densificar no se pueden unir firmas.
- **Fast Similarity Sketching** (Dahlgaard et al., 2017): $O(n + m \log m)$ y mejor varianza, pero requiere varias pasadas (no es streaming) y cuesta cerca del doble que MinHash en conjuntos pequeños.

## La pregunta de Ertl (2017)

¿Podemos tener **una sola pasada**,
**mejor precisión en conjuntos pequeños** y
**menos tiempo** que MinHash para conjuntos grandes?

# La idea central: una permutación por elemento

$$h_j(D) = \min_{d \in D} \big( r_j(d) + \pi_j(d) \big)$$

- $r_j(d) \sim U[0, 1)$.
- $\pi_j(d)$ es una permutación aleatoria de $\{0, \ldots, m-1\}$, generada por elemento con Fisher–Yates; $h_j(D) \in [0, m)$.
- Los valores $r_j(d) + \pi_j(d)$ siguen siendo uniformes e independientes entre elementos.

**Ejemplo con $m = 3$**

| | $j = 0$ | $j = 1$ | $j = 2$ |
|---|---|---|---|
| $d_1$ | 2,2 | 0,2 | 1,2 |
| $d_2$ | 1,5 | 2,5 | 0,5 |
| $d_3$ | 0,7 | 1,7 | 2,7 |
| $h(D)$ | **0,7** | **0,2** | **0,5** |

Cada fila es $r_j(d) + \pi_j(d)$ para un elemento; la firma es el mínimo por columna.

## Consecuencia

$P(h_j(A) = h_j(B)) = J$: el estimador sigue siendo insesgado, pero las entradas ya **no** son independientes.

# De la definición al algoritmo eficiente

1. **Algoritmo directo:** para cada elemento, generar la permutación completa y actualizar las $m$ posiciones. Sigue siendo $O(nm)$.
2. **Fusionar los bucles y evitar reinicializar:** mantener $p$ (la permutación actual) y $q$ (una marca del elemento que inicializó cada posición). Así no se reconstruye $p$ desde cero en cada iteración.
3. **Usar la permutación inversa:** actualizar $h_{p_j}$ en lugar de $h_j$. Como $r + j$ es estrictamente creciente en $j$, los candidatos a mejora aparecen en orden.
4. **Salida temprana:** mantener un histograma $b$ de las partes enteras de las $h_j$ y el mayor índice no vacío $a$. Si $j > a$, ninguna $h_j$ puede mejorar: se abandona el bucle interno.

## Resultado

El **Algoritmo 4** del paper. Para $n \gg m$, el número esperado de iteraciones internas por elemento tiende a $\mathbf{1}$.

# Algoritmo 4: el núcleo del método

**Entrada:** $d_0, \ldots, d_{n-1}$; tamaño de firma $m$ · **Salida:** $(h_0, \ldots, h_{m-1}) \in [0, m)^m$

$(h_0, \ldots) \leftarrow (\infty, \ldots)$; $(p_0, \ldots) \leftarrow (0, \ldots, m-1)$; $(q_0, \ldots) \leftarrow (-1, \ldots)$
$(b_0, \ldots, b_{m-1}) \leftarrow (0, \ldots, 0, m)$; $a \leftarrow m-1$
**for** $i \leftarrow 0$ **to** $n-1$ **do**
  sembrar el PRNG con $d_i$; $j \leftarrow 0$
  **while** $j \leq a$ **do**
    $r \sim U[0,1)$; $k \sim U\{j, \ldots, m-1\}$
    **if** $q_j \neq i$ **then** $q_j \leftarrow i$;
    $p_j \leftarrow j$
    **if** $q_k \neq i$ **then** $q_k \leftarrow i$;
    $p_k \leftarrow k$
    intercambiar $p_j$ y $p_k$
    **if** $r + j < h_{p_j}$ **then**
      $j' \leftarrow \min(\lfloor h_{p_j} \rfloor, m-1)$; $h_{p_j} \leftarrow r + j$
      **if** $j < j'$ **then**
        $b_{j'} \leftarrow b_{j'} - 1$; $b_j \leftarrow b_j + 1$
        **while** $b_a = 0$ **do**
          $a \leftarrow a - 1$
    $j \leftarrow j + 1$
**retorna** $(h_0, \ldots, h_{m-1})$

# Ejemplo: mínimos y corte temprano

**Firma de tamaño** $m = 3$**:** $h = [h_0, h_1, h_2]$. En cada iteración se prueba $r + j$ en la posición elegida por la permutación; se conserva el mínimo. Los valores son una ejecución ilustrativa.

**Primer elemento:** inicio $h = [\infty, \infty, \infty]$.

| $j$ | $r + j$ | Posición | Resultado |
|---|---|---|---|
| 0 | 0,4 | $h_2$ | $h_2 \leftarrow 0,4$ |
| 1 | 1,2 | $h_0$ | $h_0 \leftarrow 1,2$ |
| 2 | 2,7 | $h_1$ | $h_1 \leftarrow 2,7$ |

Firma: $[1,2;\ 2,7;\ 0,4]$; partes enteras $[1,2,0]$, por tanto $a = 2$.

**Siguiente elemento:**

- $j = 0$: candidato 0,1 actualiza $h_1$; firma $[1,2;\ 0,1;\ 0,4]$, así que $a = 1$.
- $j = 1$: candidato 1,6 no mejora $h_2 = 0,4$.
- $j = 2 > a$: para y omite las iteraciones restantes.

**Tercer elemento:** $j = 0$ actualiza $h_0$ a 0,2; ahora $a = 0$, por lo que se omiten $j = 1, 2$.

## Ahorro en esta ejecución

3 elementos $\times$ 3 iteraciones posibles $= 9$; SuperMinHash realiza $3+2+1 =$ **6 iteraciones**.

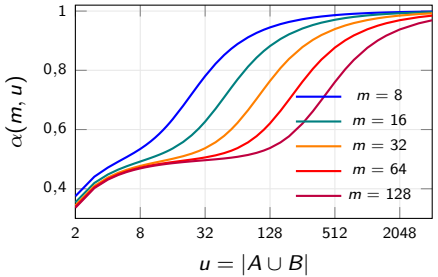
El histograma mantiene $a$ sin volver a recorrer todas las posiciones. Este ejemplo ilustra el mecanismo; no garantiza ese ahorro en cada conjunto.

# Precisión: menos varianza para conjuntos pequeños

$$\widehat{J} = \frac{1}{m}\sum_{j=0}^{m-1} \mathbf{1}[h_j(A) = h_j(B)], \qquad \mathrm{Var}(\widehat{J}) = \frac{J(1-J)}{m}\,\alpha(m,u)$$

$$\alpha(m,u) = 1 - \frac{\sum_{l=1}^{m-1} l^u\left((l+1)^u + (l-1)^u - 2l^u\right)}{(m-1)^{u-1}\, m^u\,(u-1)}, \qquad u = |A \cup B|$$

- $\alpha(m,u) \in [0,1)$: factor de reducción frente a MinHash.
- Sólo depende de $m$ y de $u = |A \cup B|$; **no** de $J$.
- Si $u < m$, la varianza es aproximadamente la mitad.
- Si $u \gg m$, $\alpha \to 1$: la ventaja desaparece.
- La mejora viene de la dependencia entre entradas, no de más hashes.



# Runtime y comparación con el estado del arte

| **Algoritmo** | **Tiempo** | **Pasadas** | **Conjuntos pequeños** | **Streaming** |
|---|---|---|---|---|
| MinHash | $O(nm)$ | 1 | referencia | sí |
| One Permutation Hashing | $O(n+m)$ | 1 | menor precisión | sí |
| Fast Similarity Sketching | $O(n+m\log m)$ | varias | mejor | no |
| **SuperMinHash** | $O(n+m\log^2 m)$ | **1** | $\alpha(m,u)<1$ | **sí** |

- $T(n,m) = n + \sum_{s=1}^{m-1} \text{mín}(t_s, n)$: $t_s$ es el tiempo de coupon collector con grupos de tamaño $s$.
- Cota: $O(n+m\log^2 m)$. Amortizado: $\to 1$ iteración por elemento cuando $n \to \infty$.
- Caso peor: $nm$; los duplicados ordenados pueden activarlo.

**En una frase**

Nunca usa más iteraciones internas que MinHash; para $n$ grande, usa muchas menos. El costo por iteración es mayor, pero la ventaja asintótica domina.

# La evidencia experimental del paper

## Precisión (Fig. 1)

- 100 000 simulaciones por caso.
- Casos con $u = 3 \cdot 2^k$ y $u = 4 \cdot 2^k$, $k = 0, \ldots, 11$.
- Se estima la varianza y se divide por la de MinHash para obtener $\alpha$ experimental.
- Las cruces coinciden con la fórmula (7).

## Runtime (Fig. 2)

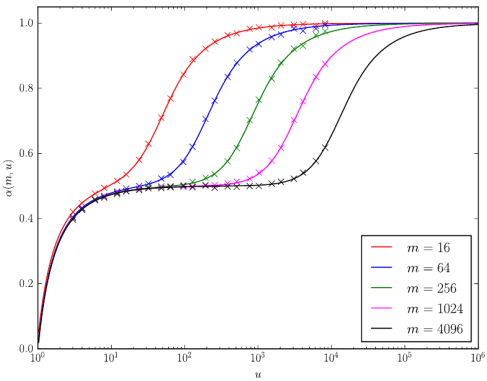
- 1 000 conjuntos aleatorios.
- $n = 2^k$, $k = 0, \ldots, 20$, y $n = 3 \cdot 2^k$, $k = 0, \ldots, 19$.
- Iteraciones internas por elemento: empiezan en $m$ y tienden a 1 cuando $n$ crece.
- Para $n$ grande, la mejora puede acercarse a un factor $m$.

## Implementación de los experimentos

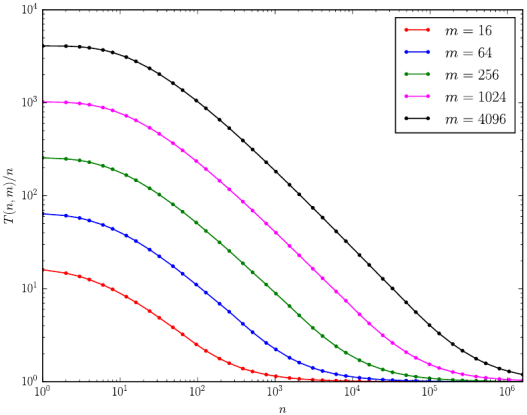
MurmurHash3 de 128 bits con semilla; secuencia predefinida de semillas para generar bits pseudoaleatorios.

El paper valida la teoría con simulación; no reporta un benchmark end-to-end de un pipeline de deduplicación.

# Gráficos de los experimentos




**Precisión:** factor $\alpha(m, u)$ de varianza relativa. Las cruces son resultados de simulación; las curvas, la predicción teórica.




**Runtime:** iteraciones internas promedio por elemento; disminuyen hacia 1 a medida que crece el conjunto.

Figuras 1 y 2 del paper de Ertl (2017).

# Fortalezas, limitaciones y crítica

- **Fortaleza:** reemplazo directo de MinHash: mismo estimador, misma interfaz, una sola pasada y mejor precisión en conjuntos pequeños.
- **La dependencia es arma de doble filo:** mejora la varianza, pero el análisis clásico de LSH *banding* supone entradas independientes. La curva-S puede desviarse; hay que calibrarla empíricamente o generar candidatos con MinHash.
- **Costo oculto:** cada iteración usa PRNG y enteros aleatorios; contar iteraciones no es lo mismo que medir tiempo de pared.
- **Duplicados:** no mejoran el estado; con duplicados ordenados el costo puede volver al caso peor $nm$.
- **Tamaño:** la ganancia de precisión es para $u < m$; si $u \gg m$, $\alpha \to 1$ y sólo queda la mejora de runtime.

## Para nuestro proyecto

Usarlo como firma principal y comparar contra MinHash; calibrar LSH con firmas dependientes y medir tiempo real, no sólo iteraciones.

# Conclusiones

1. **Mismo estimador, mejor varianza:** $\mathrm{Var}(\widehat{J}) = \frac{J(1-J)}{m}\alpha(m,u)$, con $\alpha < 1$ y aproximadamente $1/2$ para $u < m$.
2. **Mejor runtime:** $O(n + m\log^2 m)$ para elementos distintos, con costo amortizado cercano a una iteración interna por elemento.
3. **Una sola pasada:** apto para streaming y para datos que no caben en memoria.
4. **No es magia:** la ganancia depende de la calidad del hash, del PRNG, de los duplicados y del régimen de tamaños.

## Idea para recordar

SuperMinHash no calcula más hashes: cambia la forma de construir la firma para que las entradas se ayuden entre sí y para poder cortar el cómputo antes.

# Referencias I



O. Ertl. *SuperMinHash — A New Minwise Hashing Algorithm for Jaccard Similarity Estimation.* arXiv:1706.05698 [cs.DS], 2017.


A. Z. Broder. *On the Resemblance and Containment of Documents.* Compression and Complexity of Sequences (SEQUENCES), 1997.


P. Li, A. Owen, C. Zhang. *One Permutation Hashing.* Advances in Neural Information Processing Systems (NeurIPS), 2012.


S. Dahlgaard, M. B. T. Knudsen, M. Thorup. *Fast Similarity Sketching.* arXiv:1704.04370, 2017.


J. Lumbroso. *Optimal Discrete Uniform Generation from Coin Flips, and Applications.* arXiv:1304.1916, 2013.

# Backup: de la covarianza a $\alpha(m, u)$

Para $j \neq k$, el paper deriva

$$\mathrm{Cov}\big(\mathbf{1}[h_j(A) = h_j(B)],\, \mathbf{1}[h_k(A) = h_k(B)]\big) = J(1-J)\left(-\frac{1-\alpha(m,u)}{m-1}\right).$$

Para $j = k$, la covarianza es $J(1-J)$. Entonces

$$\mathrm{Var}(\widehat{J}) = \frac{1}{m^2}\left[mJ(1-J) - m(m-1)J(1-J)\frac{1-\alpha}{m-1}\right] = \frac{J(1-J)}{m}\alpha.$$

La dependencia entre entradas hace que las covarianzas sean negativas y reduzcan la suma.

# Backup: detalles de implementación

- **PRNG:** se siembra con cada elemento $d_i$; se consumen bits de MurmurHash3 de 128 bits para generar uniformes y permutaciones.
- **Permutación:** Fisher–Yates; sólo se generan las posiciones que el corte temprano necesita.
- **Invariantes:** $p$ es la permutación actual, $q_k = i$ indica que $p_k$ ya fue inicializado para $d_i$, y $b$ cuenta las partes enteras de las $h$.
- **Corte:** $a$ es el mayor índice con $b_a > 0$; si $j > a$, no hay actualizaciones posibles.
- **Duplicados:** si $d_i = d_{i'}$, la segunda inserción no cambia el estado; conviene deduplicar antes si el orden es adverso.